

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

Annexure A

Debugging or Optimization task

Code of Conduct:

I ____G. Santosh Kumar____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G. Santosh Kumar

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.

You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB

- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Q6. Producer–Consumer Debugging (Semaphore Error)

A producer-consumer application occasionally hangs after running for several minutes.

Semaphore empty = 5

Semaphore full = 0

Semaphore mutex = 1

Producer:

wait(mutex)

wait(empty)

insert(item)

signal(full)

signal(mutex)

Consumer:

wait(mutex)

wait(full)

remove(item)

signal(empty)

signal(mutex)

The program freezes even though both producer and consumer threads are alive.

1. Identify the synchronization error causing the hang.
2. Explain why the semaphore ordering is incorrect.
3. Suggest the correct semaphore sequence to eliminate the problem.

(10 Marks)

Q7. Thread Performance Debugging

A web server creates one new thread for every incoming client request.

Performance statistics:

- Average requests/sec = 15,000
- Average thread creation time = 0.4 ms
- CPU utilisation = 92%
- Memory usage continuously increases

Developers notice that the server becomes slow after several hours.

1. Identify the thread-management problem.
2. Explain why the current multithreading model causes performance degradation.
3. Propose two optimisations to improve scalability.

(10 Marks)

Q8. Banker's Algorithm Debugging

Available Resources:

A B C = 2 1 1

Allocation:

P1: 1 0 1

P2: 0 1 0

P3: 1 1 1

Maximum:

P1: 2 1 2

P2: 1 2 1

P3: 3 2 2

Process P2 requests (1,1,1).

1. Calculate the Need matrix.
2. Determine whether the request can be granted immediately using Banker's Algorithm.
3. If not, identify the unsafe condition and explain why granting the request could lead to deadlock.

(10 Marks)

Q9. Deadlock Detection and Recovery Debugging

A database server reports that four transactions are waiting for resources.

Resource Allocation Graph:

- T1 waits for R2
- T2 waits for R3
- T3 waits for R4
- T4 waits for R1
- R1 allocated to T1
- R2 allocated to T2
- R3 allocated to T3
- R4 allocated to T4

The database becomes completely unresponsive.

1. Identify whether a deadlock exists.
2. Explain how deadlock detection identifies this situation.
3. Suggest two recovery techniques and justify which one minimises system disruption.

(10 Marks)

Q10. Monitor-Based Synchronization Debugging

A monitor is used to synchronize access to a shared printer.

```
monitor Printer
{
    boolean busy = false;

    procedure Print()
    {
        if(busy)
            wait();

        busy = true;

        // Printing

        busy = false;
    }
}
```

Users report that some print jobs never execute even though the printer becomes free.

1. Identify the logical error in the monitor implementation.
2. Explain why waiting processes are never awakened.
3. Modify the monitor logic to ensure correct synchronization and fairness.

(10 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging(25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (100)
Q1						
Q2						
Q3						
Q4						
Q5						
Q6						
Q7						

Q8						
Q9						
Q10						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Operating System - CO2 AY3

Debugging

Name :- P. Parthoj Kumar
Regno :- 192529020
course code :- CS A0305

Q1. Deadlock Debugging (code trace)

Given code:-

process P1:	process P2:
lock(R1)	lock(R2)
wait(100ms)	wait(100ms)
lock(R2)	lock(R1)
critical section	critical section
unlock(R2)	unlock(R1)
unlock(R1)	unlock(R2)

- P1 holds R1 & waiting for R2, P2 holds R2 & waiting for R1. This will create an circular wait condition, so deadlock occurs

Time	process P1	process P2
t ₁	locks R1	locks R2
t ₂	waits 100ms	waits 100ms
t ₃	Tries to lock R2	Tries to lock R1
t ₄	R2 is already locked by P2 → waits	R1 is already locked by P1 → waits

- Neither process releases its first resource.
- so deadlock occurs.

It also satisfies all four Coffman deadlock conditions:

1. mutual Exclusion
only one process of resource use at a time.

2. Hold & wait

Each process holds one resource while requesting another.

3. No preemption

Resources are cannot be forcibly taken away.

4. circular wait

$P_1 - R_2 - P_2 - R_1 - P_1$

Since all these four conditions satisfies, dead lock occurs

Correct code:-

process P_1 :

lock(R_1)

lock(R_2)

critical section

unlock(R_2)

unlock(R_1)

process P_2 :

lock(R_1)

lock(R_2)

critical section

unlock(R_2)

unlock(R_1)

Q2) CPU utilisation problem

- Here CPU utilisation has only 30% even the process are ready.
- The prog is ready $\rightarrow$ waiting $\rightarrow$ ready, so the process is entered in waiting state.
- They are waiting for the I/O operations, so it is not using CPU so CPU stays most of the time idle.
- To solve this use multiprocessing then another process use the CPU time, so CPU not stays in idle.
- we can use scheduling algorithm such as multilevel feedback queue (MLFQ).

Q3. Memory Bottleneck Debugging

Given,

code = 5 MB

Heap = 150 MB

stack = 1 MB per thread

85 threads running

physical ram = 2 GB

The program experiences frequent page faults, this causes to slow down the performance, data is repeatedly uses the memory.

causes are:-

- Having too many threads uses more memory.
- frequent page faults reduces execution speed.

To overcome this problem use thread pool or reduce number of threads or Increase the RAM.

Q4. Starvation Debugging

- process P5 in Q3 (Batch queue)
- Higher priorities are Q1 & Q2 keep getting CPU time.
- Hence P5 never get a chance to execute.

Cause:-

- Continuously executing of the higher priorities processes causes the starvation of the low-priority process.

To overcome this process, use Aging to gradually increase the P5 process, multilevel feedback queue to move long-waiting processes to higher priority queue.

Q5. Context-switch Overhead Debugging

- The Server performs 20,000 context switches/sec
- It having small time quantum $\sim 1\text{ms}$
- Having high I/O frequency.

causes are :-

- Having very small Quantum time.
- Having higher I/O frequency to excessive context switching.
- CPU spends much time switching b/w the threads to execute them.

Optimization :-

- Increase the time quantum to reduce unnecessary context switching.
- Use thread pool or optimize scheduling to reduce switching overhead.

Q6. producer-consumer Debugging (semaphore error)

- The producer locks mutex before checking empty.
- The producer locks mutex before checking full.
- If the buffer is full, one process waits while holding the mutex.
- The other process cannot enter to critical section, so the program hangs.

Causes are:-

- Incorrect Semaphore ordering.
- mutex is acquired before empty or full, causing deadlock.

correct semaphore sequence:-

producer:-

wait(empty)

wait(mutex)

insert(item)

Signal(mutex)

Signal(full)

consumer:-

wait(full)

wait(mutex)

remove(item)

Signal(mutex)

Signal(empty)

The program hangs because the mutex Semaphore is locked before empty or full, causing the one thread to wait & holding the mutex.

27. Thread performance Debugging

- The server creates one new thread for every incoming client request.
- with 15,000 requests/sec having too many requests are created for threads.
- this increases memory & CPU usage likely upto 92%.

Causes:-

- Having idle threads causes high CPU overhead & memory consumption.
- Creating & destroying threads repeatedly reduces the performance.

optimization:-

1. use a thread pool to reuse existing threads.
2. limit the max no. of threads to reduce CPU & memory usage.

Q8. Banker's Algorithm debugging

Need = Max - Allocation

process	Need		
	A	B	C
P ₁	1	1	1
P ₂	1	1	1
P ₃	2	1	1

Available Resources are = 2 1 1

P₂ request are = 1 1 1

Available = 1 0 0

The system cannot find a safe sequence, so it enters to an unsafe state.

Causes:-

- Granting P₂'s request leaves an insufficient resource for the remaining processes.
- This may lead to deadlock.

final need matrix are

P₁: (1 1 1)

P₂: (1 1 1)

P₃: (2 1 1)

2. Hold & wait

Each process holds one resource while requesting another.

3. No preemption

Resources are cannot be forcibly taken away.

4. Circular wait

$P_1 - R_2 - P_2 - R_1 - P_1$

Since all these four conditions satisfies, dead lock occurs

Correct code:-

process P_1 :

lock(R_1)

lock(R_2)

critical section

unlock(R_2)

unlock(R_1)

process P_2 :

lock(R_1)

lock(R_2)

critical section

unlock(R_2)

unlock(R_1)

Q2) CPU utilisation problem

- Here CPU utilisation has only 30% even the process are ready.
- The prog is ready $\rightarrow$ waiting $\rightarrow$ ready, so the process is entered in waiting state.
- They are waiting for the I/O operations, so it is not using CPU so CPU stays most of the time idle.
- To solve this use multiprocessing then another process use the CPU time, so CPU not stays in idle.
- we can use scheduling algorithm such as multilevel feedback Queue (MLFQ).

Qno. Monitor-Based Synchronization

- The monitor uses `wait()` when the printer is busy.
- After printing is completed, the monitor does not call `signal()` or `wakeup()`.

Causes:-

- missing `signal()` after the printer becomes free.
- waiting processes remain blocked forever.

Correct code

monitor printer

boolean busy = false;

procedure print() {

while (busy)

wait();

busy = true;

print();

signal();

}

